

title: "Monde Magique — Guide de cours (débutants)"

author: "Support pédagogique"

date: "Mai 2026"

lang: fr

Monde Magique — Guide de cours (débutants)

Ce document explique **pas à pas** comment fonctionne le site/jeu **Monde Magique**, avec des **détails**, des **exemples**, et des **exercices**.

Objectif: permettre à un élève débutant de comprendre un projet web complet (front + back + base de données) et de pouvoir l'installer, le tester, le modifier.

> Projet dans ce workspace: `c:/xamppppp/htdocs/monde-magique/`

نسخة سهلة للطلاب (شرح بالعربية — مستوى مبتدئ جداً)

إذا كان الطالب لا يعرف شيئاً عن البرمجة، اتّبع هذا المسار أولاً:

1. شغّل XAMPP.
2. اضغط **Start** على **Apache** و **MySQL**.
3. افتح المتصفح واكتب:
 - `http://localhost/monde-magique/`
4. افتح صفحة التسجيل/الدخول (حسب النسخة):
 - `auth-login.php` أو صفحات `inscription.html` / `login.html`
5. سجّل حساب جديد ثم ادخل.
6. بعد الدخول ستصل إلى لوحة المراحل (Dashboard).
7. اختر المرحلة 1 ثم اضغط ابدأ.
8. أثناء اللعب، عندما تضغط "التالي"، الموقع يحفظ تقدمك تلقائياً في قاعدة البيانات.
9. أغلق المتصفح وافتحه مرة أخرى ثم ادخل: ستجد تقدمك محفوظاً.
10. إذا ظهر خطأ: افتح (F12) ثم **Console** و **Network** لتتري المشكلة.

مصطلحات سهلة:

- المتصفح (Browser) = Chrome/Edge.
- السيرفر (Server) = Apache داخل XAMPP.

Table des matières

1. Introduction et objectifs
2. Pré-requis (XAMPP, navigateur, VS Code)
3. Installation locale pas à pas
4. Tour guidé du projet (fichiers et dossiers)
5. Concepts web indispensables (HTTP, sessions, JSON)
6. Authentification (inscription/connexion)
7. Base de données (version simple et version complète)
8. API de progression (sauvegarde/chargement)
9. Dashboard (affichage des stages)
10. Stage wrapper (injection JS et restauration d'état)
11. JavaScript: `APIDataManager` et appels `fetch`
12. Données: progression, QCM, essai, récompenses
13. Debugging (erreurs fréquentes + solutions)
14. Sécurité (bonnes pratiques)
15. Extensions: ajouter un stage, un quiz, une récompense
16. Série de TP (exercices guidés)
17. Glossaire

1) Introduction

Monde Magique est une plateforme de jeu éducatif. L'enfant se connecte, accède à un tableau de bord (dashboard), puis joue des **stages** (étapes). Chaque stage contient des **stations** (quiz, jeu, contenu) et le système sauvegarde:

- la progression (ex: étape courante),
- le score de QCM,
- les essais (texte),
- les récompenses (coins/diamants).

Le projet est **bilingue** (FR/AR) et plusieurs pages sont en arabe (RTL).

Monde Magique؟) شرح بالعربية

****Monde Magique**** هو موقع/لعبة تعليمية.

- الطفل يسجل/يدخل للحساب.
- يرى لوحة فيها مراحل (Stages).
- يدخل مرحلة، يلعب ويجيب على أسئلة (QCM) ويكتب نص (Essai) أحياناً.
- يحصل على مكافآت: عملات (Coins) وألماس (Diamonds).
- كل شيء يتم حفظه في قاعدة البيانات، يعني لا يضيع التقدم.

1.1 Ce qu'on va apprendre

- Comment un site web est structuré (HTML/CSS/JS + PHP + MySQL)
- Comment un navigateur parle au serveur (requêtes HTTP)
- Comment gérer une connexion utilisateur (sessions PHP)
- Comment stocker des données (base MySQL)
- Comment faire une API simple (endpoints PHP)

2) Pré-requis

2.1 Logiciels

- Windows
- XAMPP (Apache + MySQL)
- VS Code
- Un navigateur (Chrome/Edge)

شرح بالعربية (المتطلبات)

- Windows

• برنامج XAMPP (يحتوي Apache + MySQL)

• VS Code (لفتح الملفات والتعديل)

• متصفح (Chrome أو Edge)

2.2 Vocabulaire minimum

- **Frontend**: tout ce qui est exécuté dans le navigateur (HTML/CSS/JS).
- **Backend**: code côté serveur (ici PHP).
- **API**: un script qui reçoit des paramètres et renvoie des données JSON.
- **Session**: mémoire côté serveur pour savoir qui est connecté.

شرح بالعربية (مصطلحات)

- **Frontend** (واجهة): ما تراه في المتصفح (HTML/CSS/JS).
- **Backend** (خلفية): كود يعمل على السيرفر (PHP).
- **API**: صفحة PHP تستقبل طلب وترجع جواب JSON.
- **Session** (جلسة): طريقة يعرف بها السيرفر أن المستخدم "داخل" حسابه.

3) Installation locale pas à pas (XAMPP)

3.1 Démarrer Apache et MySQL

1. Ouvrir XAMPP Control Panel
2. Cliquer sur **Start** pour **Apache**
3. Cliquer sur **Start** pour **MySQL**

شرح بالعربية (التشغيل خطوة بخطوة)

1. افتح XAMPP Control Panel.
2. اضغط Start على Apache.
3. اضغط Start على MySQL.
4. افتح المتصفح واكتب:
 • `http://localhost/monde-magique/`
5. إذا لم تفتح الصفحة، تأكد أن Apache يعمل (لون أخضر) وأن المنفذ 80 غير محجوز.

3.2 URL de test

Dans le navigateur:

- `http://localhost/monde-magique/`

3.3 Vérifier la base

Le projet peut créer automatiquement une base simple (selon `config.php`).

- Base simple: `mondo\_magique` (auto-crétation)

Il existe aussi un fichier SQL complet:

- `database/COMPLETE\_DATABASE.sql` (base riche: `monde\_magique`)

Dans ce guide, on explique **les deux**.

Option A (débutants) — base simple auto-crée

Ouvrir:

- `http://localhost/monde-magique/test-database.php` (si vous l'utilisez)
- puis s'inscrire / se connecter.

Option B (avancé) — importer la base complète

1. Ouvrir `http://localhost/phpmyadmin`
2. Importer le fichier `database/COMPLETE\_DATABASE.sql`
3. Vérifier que la base `monde\_magique` existe

> Attention: le code PHP principal utilise `mondo\_magique`. Si vous importez `monde\_magique`, il faudra soit adapter `config.php`, soit garder les deux bases.

شرح بالعربية (ملاحظة مهمة عن قاعدة البيانات)

يوجد اسمان لقواعد البيانات في المشروع:

- قاعدة بسيطة اسمها: `mondo\_magique` (ينشئها `config.php` تلقائياً)
- قاعدة كاملة اسمها: `monde\_magique` (تأتي من ملف SQL كامل)

للدراسة للمبتدئين: استعمل القاعدة البسيطة أولاً لأنها تعمل بسرعة.

4) Tour guidé du projet (fichiers importants)

Voici les fichiers essentiels:

- `index.html` : page d'accueil
- `login.html` / `inscription.html` : pages front (selon version)
- `auth-login.php` : page login/inscription (version PHP intégrée)
- `auth.php` : API d'authentification
- `config.php` : connexion DB + création de tables (version simple)
- `dashboard-stages.php` : tableau de bord des stages
- `stage.php` : wrapper qui sert `stage-\*.html` et injecte `api-manager.js`
- `api-manager.js` : gestionnaire des appels API (JS)
- `progress\_api.php` : API progression (sauvegarde, QCM, essai)
- `database/COMPLETE\_DATABASE.sql` : schéma complet (production)

Dossiers fréquents:

- `assets/`, `css/`, `js/` : ressources
- `stages/` ou `stage-\*.html` : contenus des stages

شرح بالعربية (ملفات مهمة جدًا)

- `index.html` : الصفحة الرئيسية.
- `auth-login.php` : صفحة الدخول/التسجيل (PHP).
- `auth.php` : API للدخول/التسجيل والجلسات.
- `config.php` : اتصال قاعدة البيانات وإنشاء الجداول (نسخة بسيطة).
- `dashboard-stages.php` : لوحة المراحل.
- `stage.php` : يفتح ملف المرحلة ويضيف كود الحفظ.
- `api-manager.js` : كود JavaScript الذي يرسل البيانات إلى الـ API.
- `progress\_api.php` : API لحفظ/تحميل التقدم ونتائج الأسئلة.

5) Concepts web indispensables

5.1 HTTP en 2 minutes

Le navigateur envoie une requête:

- GET (récupérer une page ou des données)
- POST (envoyer des données)

Exemples:

- `GET /monde-magique/dashboard-stages.php`
- `POST /monde-magique/auth.php?action=login`

GET و POST (فكرة) شرح بالعربية

- **GET** يعني: "أعطني بيانات/صفحة".
- **POST** يعني: "سأرسل لك بيانات لتحفظها".

مثال:

- GET لفتح لوحة المراحل.
- POST عند إرسال اسم المستخدم وكلمة السر.

5.2 JSON

Le JSON est un format texte pour envoyer des données:

```
{
  "success": true,
  "user_id": 12,
  "username": "demo"
}
```

JSON؟ ما هو) شرح بالعربية

JSON هو شكل منظم لإرسال البيانات بين المتصفح والسيرفر.

- يشبه جدول/قاموس.
- سهل القراءة.
- API عادة ترجع JSON.

5.3 Sessions PHP

Une **session** permet de dire: "cet utilisateur est connecté".

- Le serveur stocke `\$\_SESSION['user\_id']`
- Le navigateur garde un cookie de session

Dans le code, beaucoup de pages font:

```
session_start();
if (!isset($_SESSION['user_id'])) {
    header('Location: auth-login.php');
    exit;
}
```

Session؟ ما هي) شرح بالعربية

Session تعني: السيرفر يتذكر أن هذا المستخدم “مسجل دخول”.

- إذا لم توجد Session، الموقع يرجعك لصفحة الدخول.
- Session تُحفظ في السيرفر، والمتصفح يرسل Cookie صغير للتعريف.

6) Authentication (inscription/connexion)

6.1 Le fichier `auth.php`

`auth.php` est une API PHP qui gère plusieurs actions via un paramètre `action`:

- `action=register`
- `action=login`
- `action=logout`
- `action=check\_session`
- `action=get\_user\_data`

Le principe:

1. Le frontend envoie un POST
2. PHP valide
3. PHP écrit en base
4. PHP met `\$\_SESSION['user\_id']`
5. PHP renvoie un JSON

شرح بالعربية (التسجيل والدخول)

`auth.php` هو ملف PHP يعمل كـ API.

- `register`: تسجيل حساب جديد.
 - `login`: دخول.
 - `logout`: خروج.
 - `check\_session`: يتأكد هل المستخدم داخل أم لا.
- عند التسجيل:

1. المتصفح يرسل البيانات.
2. السيرفر يتحقق منها.
3. يحفظها في قاعدة البيانات.
4. يفتح Session.

6.2 Inscription (register)

Pendant l'inscription, le backend:

- vérifie les champs
- hash le mot de passe avec `password\_hash(..., PASSWORD\_BCRYPT)`
- crée l'utilisateur
- crée 10 lignes de progression (stages 1..10)
- crée 1 ligne de récompenses

Exemple: requête (pseudo-code JS)

```
await fetch('auth.php?action=register', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  credentials: 'include',
  body: JSON.stringify({
    username: 'eleve1',
    email: 'eleve1@example.com',
    password: 'secret123',
    password_confirm: 'secret123'
  })
});
```

6.3 Connexion (login)

Le backend:

- cherche l'utilisateur par username ou email
- vérifie le mot de passe avec `password\_verify`
- met à jour `last\_login`
- ouvre une session

Exercice 1

1. Créer un compte
2. Se déconnecter
3. Se reconnecter
4. Observer: `dashboard-stages.php` vous reconnaît

7) Base de données

شرح بالعربية (قاعدة البيانات ببساطة)

قاعدة البيانات هي مكان حفظ المعلومات:

- حسابات المستخدمين
- تقدم كل مرحلة
- النقاط والجوائز

إذا لم نحفظ في قاعدة البيانات، قد تضيع المعلومات عند إغلاق المتصفح.

Le projet a **deux niveaux**:

1. **Base simple** auto-créeée par `config.php` (pour démarrer vite)
2. **Base complète** `database/COMPLETE\_DATABASE.sql` (production / très riche)

7.1 Base simple (créée par `config.php`)

Base: `mondo\_magique`

Tables:

- `users`
- `progress`
- `rewards`
- `qcm\_answers`
- `essays`

7.1.1 Table `users`

Contient l'identifiant, username, email, password hash, dates.

7.1.2 Table `progress`

Une ligne par (user, stage).

Champs importants:

- `stage\_num`
- `completed`
- `qcm\_score`
- `essay\_score`
- `diamonds`, `coins`
- `last\_step`

Exemple SQL (lecture)

```
SELECT * FROM progress WHERE user_id = 1 ORDER BY stage_num;
```

7.2 Base complète (fichier SQL)

Base: `monde\_magique`

Elle contient beaucoup plus:

- profils, settings JSON
- `stages`, `stations`, `quiz\_questions`, `quiz\_results`
- économie: `shop\_items`, `transactions`
- social: `friends`, `friend\_requests`
- logs: `activity\_logs`

- chatbot: `chatbot\_conversations`, `chatbot\_logs`

Pourquoi c'est intéressant pédagogiquement?

- Vous apprenez un schéma "réaliste" d'un jeu web
- Vous voyez l'usage de JSON en base (MySQL JSON)
- Vous comprenez l'évolution "prototype → production"

Exercice 2

Dans phpMyAdmin:

1. Ouvrir la table `stages`
2. Trouver l'étape `tunisia`
3. Changer son titre (attention à ne pas casser le code)

8) API de progression (`progress\_api.php`)

(التقدم؟ API لماذا) شرح بالعربية

المرحلة (Stage) عبارة عن صفحة HTML/JS.

عندما يتقدم الطالب في المرحلة، نحتاج أن نرسل "التقدم" للسيرفر ليتم حفظه.

`progress\_api.php` هو الملف الذي يستقبل هذه الطلبات ويحفظها في MySQL.

8.1 Pourquoi une API?

Les stages sont des pages HTML/JS. Quand l'élève avance dans le stage, on doit ****sauvegarder**** la progression sur le serveur.

`progress\_api.php` reçoit une action et travaille avec la base.

Exemples d'actions:

- `save\_progress` (POST)
- `load\_progress` (GET)
- `load\_all\_progress` (GET)
- `save\_qcm` (POST)

- `save\_essay` (POST)
- `complete\_stage` (POST)
- `get\_rewards` (GET)

8.2 Format des données

Deux styles existent dans le projet:

- Certaines fonctions lisent `php://input` (JSON)
- D'autres attendent des `FormData` (POST classique)

Il faut donc faire attention: ****le frontend doit envoyer le format attendu****.

Exemple: sauvegarder progression (FormData)

```
const fd = new FormData();
fd.append('action', 'save_progress');
fd.append('stage_num', 3);
fd.append('current_step', 2);
fd.append('qcm_score', 80);
fd.append('diamonds', 5);
fd.append('coins', 120);

await fetch('progress_api.php', {
  method: 'POST',
  body: fd,
  credentials: 'include'
});
```

9) Dashboard des stages (`dashboard-stages.php`)

شرح بالعربية (لوحة المراحل)

هذه الصفحة:

- تتأكد أنك مسجل دخول.
- تقرأ تقدمك من قاعدة البيانات.
- تعرض بطاقات المراحل.

Le dashboard:

- vérifie la session
- lit `rewards` + `progress`
- calcule combien de stages sont terminés
- affiche une grille de stages

Côté HTML, la page est en **arabe**:

- `lang="ar" dir="rtl"``

Le JavaScript reçoit des variables générées par PHP:

- `userProgress` (JSON)
- `userRewards` (JSON)

Puis il rend les cartes (stages) et construit les boutons "Play".

Exercice 3

1. Ouvrir le dashboard
2. Cliquer stage 1
3. Noter l'URL: `stage.php?stage=1`

10) Stage wrapper (`stage.php`)

stage.php? (ما وظيفة شرح بالعربية)

`stage.php` يشبه "غلاف" للمرحلة.

- يمنع فتح المرحلة بدون تسجيل دخول.
- يفتح ملف المرحلة HTML الصحيح.
- يضيف JavaScript يساعد على حفظ التقدم.

10.1 Pourquoi un wrapper?

Les fichiers `stage-1-tunisia.html`, etc. sont des pages HTML (front).

Mais on veut:

- empêcher l'accès si non connecté
- injecter `api-manager.js`
- restaurer la progression au chargement
- sauvegarder automatiquement au clic "next"

C'est ce que fait `stage.php`:

1. Vérifie la session
2. Charge le bon fichier HTML de stage
3. Lit la progression en DB
4. Injecte des scripts JS avant `</body>`

10.2 Injection (principe)

`stage.php` fait:

- `file\_get\_contents(stage-\*.html)`
- remplace `</body>` par `... + scripts + </body>`

Cela permet d'ajouter une couche API sans réécrire chaque stage.

11) JavaScript: `api-manager.js`

(الذي يحفظ التقدم JavaScript) شرح بالعربية

`api-manager.js` يرسل طلبات إلى `auth.php` و `progress\_api.php`.

مثال:

- عند الضغط "التالي": يرسل `saveProgress()` لحفظ رقم الخطوة والنتيجة.

11.1 Rôle de `APIDataManager`

La classe `APIDataManager`:

- vérifie la session via `auth.php?action=check\_session`

- mémorise `userId`
- mémorise `stageNum`
- expose des méthodes:
- `saveProgress()`
- `loadProgress()`
- `saveQCM()`
- `saveEssay()`
- `completeStage()`
- `getRewards()`

11.2 Important: cohérence des champs JSON

Quand vous créez une API, il faut une convention stricte.

Exemple:

- Backend renvoie `logged\_in: true`
- Front attend `authenticated: true`

Si ce n'est pas identique, le front peut croire que l'utilisateur n'est pas connecté.

> Dans un TP "debug", on apprendra à repérer ce type de bug.

12) Données: progression, QCM, essai, récompenses

شرح بالعربية (ما الذي يتم حفظه؟)

- **Progression**: أين وصلت داخل المرحلة.
- **QCM**: إجابات الاختيار من متعدد والنتيجة.
- **Essai**: النص الذي يكتبه الطالب وعدد الكلمات.
- **Récompenses**: العملات والألماس.

12.1 Progression

Progression = où en est l'élève dans le stage.

Exemples de champs:

- `last\_step` ou `current\_step`
- `completed`

12.2 QCM

Le QCM a:

- les réponses de l'utilisateur (A/B/C/D)
- quelles questions sont correctes
- un score

12.3 Essai (texte)

Un essai est sauvegardé avec:

- `content`
- `word\_count`
- `score`

Le score peut être calculé simplement (ex: basé sur nombre de mots).

12.4 Récompenses

Récompenses totales dans `rewards`:

- `total\_diamonds`
- `total\_coins`
- `total\_stages\_completed`

13) Debugging: erreurs fréquentes

شرح بالعربية (أخطاء شائعة)

- 401 غير مُصادق: يعني لم يتم تسجيل الدخول أو لم تُرسل Session.
- خطأ JSON: غالبًا بسبب خطأ/تحذير في PHP قبل طباعة JSON.
- مشكلة قاعدة البيانات: اسم قاعدة مختلف أو MySQL متوقف.

13.1 Erreur 401 “Non authentifié”

Cause typique:

- session pas créée
- cookie pas envoyé (manque `credentials: 'include'` en fetch)

Solution:

- vérifier `session\_start()`
- vérifier `credentials: 'include'`

13.2 Erreur JSON invalide

Cause:

- PHP affiche un warning avant le JSON

Solution:

- activer logs
- corriger warning

13.3 Base de données incorrecte

Cause:

- confusion `mondo\_magique` vs `monde\_magique`

Solution:

- choisir une base
- aligner `DB\_NAME`

14) Sécurité (bonnes pratiques)

شرح بالعربية (أمان بسيط)

- لا تخزن كلمة السر كما هي، بل استخدم `password\_hash`.

- لا تسمح لأي شخص بالدخول للصفحات الحساسة بدون Session.
- انتبه لحقن SQL (استعمل طرق آمنة للـ SQL).

14.1 Mots de passe

Toujours:

- ``password_hash`` pour stocker
- ``password_verify`` pour vérifier

14.2 SQL injection

Le projet utilise ``real_escape_string`` dans plusieurs endroits.

Pour aller plus loin, enseigner aux élèves:

- requêtes préparées (``prepare``, ``bind_param``)

14.3 Sessions

- ne jamais exposer ``user_id`` dans le HTML inutilement
- vérifier la session côté serveur sur les pages sensibles

15) Extensions: comment ajouter du contenu

15.1 Ajouter un nouveau stage

1. Créer un fichier ``stage-11-xxx.html``
2. Mettre à jour la map ``$stageFiles`` dans ``stage.php``
3. Ajouter la ligne en DB (si vous utilisez la base complète)
4. Ajouter une carte dans le dashboard

15.2 Ajouter une question QCM

Dans la base complète:

- table ``quiz_questions``

Dans la version simple:

- les réponses sont stockées dans ``qcm_answers``
-

16) Série de TP (exercices guidés)

TP 1 — Installer et lancer

- Démarrer Apache/MySQL
- Ouvrir la page d'accueil
- S'inscrire
- Accéder au dashboard

TP 2 — Lire la progression en base

- Ouvrir phpMyAdmin
- Observer la table ``progress``
- Comprendre les colonnes

TP 3 — Appels API à la main

Objectif: comprendre l'API.

- Dans la console (F12), faire un ``fetch`` manuel vers ``progress_api.php?action=load_all_progress``
- Lire la réponse

TP 4 — Débogage “champ JSON incohérent”

Objectif: comprendre pourquoi un front “redirige” vers login.

- Voir la réponse de ``auth.php?action=check_session``
- Comparer avec ce que le JS attend
- Proposer une correction

TP 5 — Petite amélioration

- Ajouter l’affichage du ``qcm_score`` sur une carte de stage dans le dashboard

17) Glossaire

- **Endpoint**: URL d’une API
- **Session**: mémoire serveur pour un utilisateur
- **Credential cookie**: cookie envoyé avec ``fetch(..., { credentials: 'include' })``
- **CRUD**: Create/Read/Update/Delete
- **InnoDB**: moteur MySQL supportant les clés étrangères

Annexes (exemples supplémentaires)

A) Exemple de diagramme de flux (texte)

1. User → Dashboard → clique Stage
2. Dashboard → ``stage.php?stage=3``
3. ``stage.php`` → injecte JS
4. JS → ``loadProgress()``
5. User joue → ``saveProgress()``
6. DB stocke → dashboard reflète progression

B) Mini-questions pour révision

1. Pourquoi on utilise une session?
2. Différence entre GET et POST?
3. Pourquoi on hash les mots de passe?
4. Que signifie 401?

18) Comprendre la structure des pages (Frontend)

Cette section est dédiée aux élèves débutants qui ne savent pas encore “lire” un projet web.

18.1 Les fichiers HTML

Le projet contient:

- des pages générales: `index.html`, `profile.html`, `shop.html`, etc.
- des pages de jeu: `stage-1-tunisia.html` ... `stage-10-world.html`

À retenir

- Une page HTML contient le contenu (titres, boutons, images)
- Le CSS rend la page jolie
- Le JavaScript donne la logique (clics, scores, transitions)

18.2 CSS: pourquoi il y a un dossier `css/`

Le CSS sert à:

- mettre des couleurs
- créer des grilles
- gérer le responsive (mobile/tablette)
- animer (transitions)

Exercice 4 (CSS facile)

1. Ouvrir une page (ex: `dashboard-stages.php`)
2. Chercher une règle CSS (ex: `.header h1`)
3. Modifier une taille de police
4. Rafraîchir la page et observer

18.3 Bilingue et RTL (arabe)

Certaines pages sont en arabe et utilisent:

```
<html lang="ar" dir="rtl">
```

Explications:

- `lang="ar"`: indique la langue

- ``dir="rtl"`` : l'écriture va de droite à gauche

Exercice 5

1. Remplacer temporairement ``dir="rtl"`` par ``dir="ltr"``
2. Observer comment l'interface change
3. Remettre ``rtl``

19) Backend PHP: comprendre ``config.php``

Le fichier ``config.php`` est inclus par les pages PHP.

Il fait 3 choses principales:

1. Se connecter à MySQL
2. Créer la base + tables si elles n'existent pas (version simple)
3. Définir des headers JSON/CORS

19.1 Connexion MySQL

Dans la version simple:

```
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', '');
define('DB_NAME', 'mondo_magique');
```

Exercice 6

1. Ouvrir phpMyAdmin
2. Vérifier si la base ``mondo_magique`` existe
3. Si oui, ouvrir ``users`` et voir les données après inscription

19.2 Auto-crédation des tables

Le tableau ``$tables_sql`` contient des instructions ``CREATE TABLE IF NOT EXISTS ...``.

Avantage (débutants):

- on n'a pas besoin d'importer un SQL pour démarrer

Inconvénient (projet réel):

- la version "simple" n'a pas toutes les tables et fonctionnalités

19.3 Headers JSON et CORS

Le fichier active:

```
header('Content-Type: application/json; charset=utf-8');  
header('Access-Control-Allow-Origin: *');
```

Attention: sur des projets réels, `Access-Control-Allow-Origin: \*` n'est pas toujours souhaité.

20) Contrat de l'API (spécification simple)

Cette section est très utile pour les élèves: elle décrit "ce que l'API promet".

20.1 API Auth (`auth.php`)

Base URL (local):

- `http://localhost/monde-magique/auth.php`

`POST auth.php?action=register`

Entrée (JSON):

```
{  
  "username": "eleve1",  
  "email": "eleve1@example.com",  
  "password": "secret123",  
  "password_confirm": "secret123"  
}
```


Sortie (succès):

```
{
  "success": "Inscription réussie",
  "user_id": 1,
  "username": "eleve1"
}
```

Erreurs possibles:

- `400` champs manquants
- `400` mot de passe trop court
- `400` utilisateur existe

`POST auth.php?action=login`

Entrée (JSON):

```
{
  "username": "eleve1",
  "password": "secret123"
}
```

Sortie (succès):

```
{
  "success": "Connexion réussie",
  "user_id": 1,
  "username": "eleve1"
}
```

`GET auth.php?action=check\_session`

Sortie possible:

```
{ "logged_in": true, "user_id": 1, "username": "eleve1" }
```

ou

```
{ "logged_in": false }
```

> Point pédagogique: le frontend doit attendre `logged\_in`, pas `authenticated`.

20.2 API Progression (`progress\_api.php`)

Base URL:

- `http://localhost/monde-magique/progress\_api.php`

`GET progress\_api.php?action=load\_progress&stage\_num=1`

Retour: un objet décrivant la progression du stage.

`GET progress\_api.php?action=load\_all\_progress`

Retour: un objet avec les stages (1..10).

`POST progress\_api.php` (FormData)

Les actions `save\_progress`, `save\_qcm`, `save\_essay`, `complete\_stage` utilisent des champs `FormData`.

21) Étude de cas: comment un stage sauvegarde sa progression

Cette section suit exactement le chemin d'un clic "next".

21.1 Au chargement du stage

1. L'utilisateur ouvre: `stage.php?stage=3`
2. `stage.php` charge `stage-3-africa.html`
3. `stage.php` injecte `api-manager.js`

4. JS appelle ``waitForSession()``
5. Puis ``apiManager.loadProgress()``
6. Le stage restaure ``currentStep`` et les ressources

21.2 Au clic sur “next”

Le wrapper redéfinit ``nextStep()`` pour:

- exécuter la logique du stage
- puis appeler ``apiManager.saveProgress(...)``

Exercice 7 (observer en vrai)

1. Ouvrir un stage
2. Appuyer F12 → onglet Network
3. Cliquer “next”
4. Observer l'appel à ``progress_api.php``
5. Lire les champs envoyés

22) Base complète: explication pédagogique du grand schéma SQL

La base complète (``database/COMPLETE_DATABASE.sql``) est un exemple “pro” de base de données de jeu.

22.1 Tables centrales

- ``users``: profils, langue, préférences, économie
- ``stages``: les 10 régions/étapes
- ``stations``: contenu et type (quiz, vidéo, chatbot...)
- ``quiz_questions`` + ``quiz_results``
- ``transactions`` + ``shop_items``

22.2 JSON en base

Exemples d'usage:

- `users.settings` (préférences)
- `stages.base\_rewards`
- `stations.questions`

Exercice 8 (lecture JSON)

Dans MySQL:

```
SELECT stage_key, JSON_EXTRACT(base_rewards, '$.coins') AS coins
FROM stages
ORDER BY stage_order;
```

22.3 Clés étrangères

Les clés étrangères imposent la cohérence:

- un `stage\_progress.user\_id` doit exister dans `users`
- si on supprime un user, ses progressions peuvent être supprimées (cascade)

Exercice 9

1. Trouver une table avec `FOREIGN KEY`
2. Comprendre `ON DELETE CASCADE`

23) Debugging avancé (atelier)

Cette partie est faite pour l'enseignant: elle crée des “bugs” éducatifs.

23.1 Bug: le JS pense que la session est invalide

Symptôme:

- redirection vers `auth-login.php`

Cause possible:

- le JS attend `data.authenticated` mais l'API renvoie `logged\_in`

Méthode:

1. Ouvrir Network
2. Lire la réponse JSON
3. Corriger la condition côté JS

23.2 Bug: l'API attend JSON mais on envoie FormData (ou inverse)

Symptôme:

- champs vides côté PHP

Méthode:

1. Vérifier si PHP lit `$_POST`
2. Vérifier si le JS envoie `Content-Type: application/json`

23.3 Bug: "Warning" PHP casse le JSON

Symptôme:

- `Unexpected token < in JSON`

Cause:

- un warning PHP est envoyé avant le JSON

Solution:

- corriger le warning
- éviter `echo` de HTML dans une API JSON

24) Séances de cours (plan enseignant)

Cette proposition donne un cursus sur 6 à 8 séances.

Séance 1 — Découverte du projet (60-90 min)

- Lancer XAMPP

- Ouvrir le site
- Visiter le dashboard
- Comprendre “frontend vs backend”

Séance 2 — Authentification et sessions

- `auth.php` actions
- `session\_start()`
- comprendre cookies
- exercice: bloquer une page si non connecté

Séance 3 — Base de données (simple)

- lire `config.php`
- explorer tables `users`, `progress`, `rewards`
- exercice: faire une requête SELECT

Séance 4 — API progression

- explorer `progress\_api.php`
- exercice: appeler `load\_all\_progress`
- comprendre codes HTTP

Séance 5 — Front JS

- `fetch`, `FormData`
- observer Network
- exercice: afficher un score sur le dashboard

Séance 6 — Mini-projet

Au choix:

- ajouter un badge simple
- ajouter une page “aide”
- ajouter une statistique

25) Corrigés (résumés)

Corrigé Exercice 1

Attendu:

- un utilisateur apparaît dans `users`
- 10 lignes apparaissent dans `progress`
- 1 ligne apparaît dans `rewards`

Corrigé Exercice 7

Attendu:

- dans Network, un POST vers `progress\_api.php`
- les champs `stage\_num`, `current\_step` évoluent

26) Bonus: commandes utiles (pour élèves)

26.1 Vérifier Apache

- URL: `http://localhost/`

26.2 Vérifier phpMyAdmin

- URL: `http://localhost/phpmyadmin`

26.3 Vérifier le projet

- URL: `http://localhost/monde-magique/`

27) Checklists

Checklist installation

- ☐ Apache démarré
- ☐ MySQL démarré
- ☐ Le site s'ouvre
- ☐ Inscription fonctionne
- ☐ Dashboard affiche stages

Checklist debugging

- ☐ Console sans erreurs critiques
- ☐ Network montre les appels API
- ☐ Les réponses API sont du JSON valide